Detecting Broken Access Control in RBAC Systems Using Specialized API Fuzzing Techniques

Seth Limbo

June 2, 2026

1 Introduction

Application Programming Interfaces (APIs) play a central role in modern software systems by enabling communication between services, applications, and users. Because they often serve as entry points to critical data and functionality, APIs have become a frequent target of attacks, with Broken Access Control (BAC) identified as one of the most widespread vulnerabilities across web applications [DAP+25]. Ensuring that APIs enforce access policies correctly is therefore essential for maintaining security in real-world deployments.

Traditional testing methods for APIs often rely on role- and policy-based approaches, where access rules such as Role-Based Access Control (RBAC) are validated against predefined specifications. Some of these methods use FSMs[DMS16], wherein an exhaustive approach has been seen to show complete proof of role-correctness, after the modelling of roles and privileges. However, the proportion between (1) the number of roles and policies and (2) testing resources (test suite length, runtime, and cost) can often prevent the complete and thorough testing of RBAC systems.

To address these gaps, the emergence of API fuzzing can be used as a complementary technique. Fuzzing is an automated testing method that generates diverse input data to uncover bugs, security flaws, and edge cases that may escape conventional policy-based validation. Fuzzing excels at scalability and automation.

This paper examines the intersection of these two perspectives. We first review how APIs and RBAC policies are implemented and tested in practice, highlighting their relevance to Broken Access Control vulnerabilities. We then discuss recent advances in API fuzzing, including state-aware, schema-driven, and feedback-based approaches. Finally, we explore how these fuzzing techniques can be extended or adapted to evaluate RBAC systems, with particular attention to BACFuzz, a framework specifically designed for detecting access control flaws.

2 APIs and RBAC

Application Program Interfaces (APIs) are the set of protocols that allows software to exchange data across software endpoints. Through these APIs, each software can authenticate users, create requests, and deliver responses about data being stored and processed by the system. However, with the application of software towards security-critical data management, enforcing limitations on user access to system data is important to secure both institutional and individual interests of secrecy and privacy, commonly through the methods of Access Control, with Role Based Access Control (RBAC) being in use for many organizations.

2.1 RESTful Application Program Interfaces

RESTful APIs follow the Representational State Transfer (REST) style, where resources are exposed over HTTP using standard methods such as GET, POST, PUT, and DELETE. Their lightweight and scalable design has made them the dominant architecture for data exchange across systems. Given

that REST APIs serve as the primary interface for security-critical applications, they provide a natural focus for examining access control and vulnerability risks in modern software systems [GZA23].

Research on RESTful APIs highlights both their importance in modern cloud applications and the challenges of ensuring their reliability. A recent survey of 92 studies shows a rapid increase in attention to REST API testing since 2017, with approaches ranging from schema-based test generation to automated fuzzing techniques [GZA23]. The study emphasizes that REST APIs are typically defined through specifications such as OpenAPI, which facilitate automated documentation, test case generation, and validation of request–response behavior. Despite progress, the survey identifies gaps in industrial adoption, limited case studies, and emerging yet underexplored areas, such as security-focused testing. These findings suggest that, while RESTful APIs are now a central focus in software engineering research, more work is needed to close the gap between academic techniques and practical application.

While the survey highlights the technical challenges of testing and validating REST APIs, another line of research focuses on strengthening their authentication and authorization mechanisms. A study on Scientific Computing Environments introduced microservice-based solutions that provided single sign-on across multiple web communities and flexible client-side authentication workflows [CWC+19]. By exposing high-performance computing resources through RESTful APIs, the system enabled secure yet simplified access for users and developers. This case illustrates how well-designed authentication services can improve testing efforts by addressing the access control dimension of API reliability, striking a balance between usability and security in real-world deployments.

2.2 Role Based Access Control

In IT systems, Access Control is a security process that evaluates the validity of user requests to stored data and resources based on a set of policies. Role-Based Access Control is a type of Access Control that is enforced by granting a collection of policies to roles that mimic a system's organization of users.

Samarati and Vimercati[SdV01] describe the implementation of Access Control into 3 layers of abstraction: Security Policies, Security Models, and Security mechanisms. Upon listing the different methods of Access Control within their study, including Role Based Access Control (RBAC), their implementations were described with respect to these abstractions. For the interest of this research, they noted that permissible actions within a Role-Based Access Control system can be grouped together to replicate the tasks needed for available roles within an organization. Like how organizations often follow or allow hierarchies of responsibilities, RBAC can organize its policies into a tree-like structure wherein a group, or hierarchy of policies, can be granted to a role, with roles being able to be granted or removed from individual users. They noted that RBAC has advantages, such as being capable of a more streamlined management for authorization and proper separation of duties. However, RBAC still had its limitations, such as when specific task delegations are not standard for users under a particular role, or when permissions are based on data objects rather than roles.

Farhadighalati et. al.[FEJNHB25], in a systematic review of Access Control models, describe a larger number of distinct access control categories and subtypes, even for Role Based Access. Specifically, for RBAC, they first identified its Flat variation, wherein a heirarchy of permissions are not necessarily implemented. This was followed by Heirarchical RBAC, as well as more advanced extensions of RBAC, or a combination of other Access Control models. Mehra and Taresh[Meh24], discusses the security advantages of implementing RBAC, such being a step towards data protection regulations, and by isolating the impact of cybersecurity attacks like malware and ransomware. The auditing of invalid requests made by users given their roles, was also made possible. The implementation of RBAC in the financial and healthcare systems was noted to be effective.

3 API Fuzzers

Fuzzing (also known as fuzz testing) works by generating large amounts of input data, feeding them into the target software, and monitoring the system response to crashes, errors, or abnormal behavior [DAT25]. Because it has become a diverse field with frameworks targeting different types of applications, fuzzing is particularly valuable in the context of APIs, which often involve complex input

structures and security-critical functions that may not be fully addressed by role- or policy-based methods. According to a survey paper, we can broadly categorize fuzzing frameworks by their level of knowledge of specifications, their handling of stateful versus stateless interactions, and the types of feedback mechanisms they have [DAT25]. The authors emphasize that modern fuzzers increasingly aim to balance scalability with precision, often using machine learning to guide test case generation. These have led to research into specialized approaches such as schema-driven fuzzers, stateful/stateless fuzzers, and feedback-guided fuzzing, which will be discussed in the following sections.

3.1 State-Aware and Schema-Driven Fuzzers

In their survey on fuzzers designed for stateful systems, Daniele et al. [DAP24] investigated how accounting for request order and dependencies impacts vulnerability discovery in APIs. Using case studies across multiple API-driven applications, they compared stateless fuzzers (which only mutate single requests) with state-aware fuzzers that must also preserve valid state transitions. The researchers found that while state-aware approaches uncovered deeper, state-dependent flaws, they also faced the challenge of state space explosion, where the number of possible request sequences grows prohibitively large.

To address these challenges, they evaluated techniques such as snapshotting (to avoid replaying long request chains) and grammar-based inference of state machines. Although these methods improved exploration efficiency, they also introduced trade-offs: snapshotting caused runtime overhead, inferred models were often incomplete, and results varied significantly between systems [DAP24]. They concluded that although state-aware fuzzing holds promise for APIs, more systematic benchmarking is needed to balance scalability with precision.

Building on this line of research, Godefroid et al. [GHP20] examined another form of fuzzing that takes advantage of REST API specifications to guide the generation of request payloads through schema-driven techniques. Rather than mutating inputs blindly, their approach treated request body schemas as tree structures and applied fuzzing rules to modify nodes and types. The goal was to intelligently generate valid yet diverse JSON payloads that could expose data processing bugs in cloud APIs.

The authors applied schema-driven fuzzing rules, sometimes combined with search heuristics and learning values from previous responses, against several Microsoft Azure services to evaluate the various techniques used. 17 unique bugs were identified by the researchers with the help of the schema-driven fuzzer during this experiment. Although the approach improved input validity and reduced wasted test cases, its effectiveness depended heavily on the completeness of API specifications, and certain trade-offs (such as relying on schema-provided examples) limited exploration of undocumented edge cases. In general, the study highlights how schema awareness enables fuzzers to uncover vulnerabilities that mutation-only approaches are likely to miss [GHP20].

3.2 Feedback mechanisms in API fuzzing

Feedback is a central element in modern fuzzing, as it guides the mutation process and determines how effectively a fuzzer can discover vulnerabilities. Dharmaadi et al. [DAT25] categorize several forms of security-relevant feedback in the context of web API fuzzing, ranging from standard HTTP response codes to more advanced metrics such as code coverage, branch distance, taint feedback, and Test Coverage Level (TCL). Although HTTP responses (e.g., 200 and 500 status codes) remain the most widely used feedback due to their availability in any web service, more sophisticated techniques, such as instrumentation-based code coverage and taint tracking, enable fuzzers to target untested paths and security-sensitive operations. These feedback channels not only improve efficiency by filtering out ineffective inputs but also expand the depth of exploration by guiding fuzzers toward hidden vulnerabilities. This overview provides the basis for understanding how newer feedback-driven approaches attempt to push fuzzing beyond the blind input mutation.

Building on this theme, a proposed approach called DocFuzz introduced a directed fuzzing method that integrates reinforcement learning with a feedback-based mutator [XZY+24]. Unlike conventional fuzzers that depend on fixed mutation rules, DocFuzz dynamically prioritizes mutations by identifying "high-value" input bytes through taint tracking. When the reinforcement learning process stagnates, the feedback mechanism reinitializes the mutator, guiding further exploration toward vulnerable code

regions. Experimental results on data sets such as LAVA-M showed that this design uncovered vulnerabilities more efficiently than state-of-the-art fuzzers, particularly for memory corruption issues.

Although code coverage is a common feedback signal in fuzzing, recent studies emphasize that its value is not consistent across all environments. An investigation showed that although coverage information generally improves guidance, its usefulness is highly dependent on surrounding conditions such as input structure, system characteristics, and parser complexity [XZY+24].In fact, achieving higher coverage does not always translate into finding more vulnerabilities, particularly when poor quality seeds limit exploration. A separate analysis of 44 real-world bugs found in OSS-Fuzz echoed this observation: while coverage feedback aided discovery in some cases, the availability of carefully selected initial inputs often contributed more to effective bug finding [JKK+24]. Together, these studies suggest that although coverage-driven fuzzing remains a strong heuristic, its benefits are context sensitive and must be weighed against the engineering costs of implementing it in specialized systems.

4 API Testing towards Broken Access Control

Broken Access Control, according to OWASP[OWA21], has emerged as the leading security issue for web applications. As such, thorough testing procedures have to be developed and maintained to ensure the validity of existing access policies. As such, vulnerability detection is a key area towards its prevention. While existing automated methods towards this are present across previous areas of research, the general use of any of these are often limited by the following: (1) need for knowledge on system implementation, (2) lack of flexibility to quickly adapt to new changes, (2) being tested or applied onto specific technologies only, (3) very high growth of the number of test cases with respect to the number of roles and policies. Nevertheless, the application of fuzzers towards BAC is a new area of research and has been shown to capable of catching Broken Function Level Authorization and Broken Object Level Authorization.

4.1 Broken Access Control

Broken Access Control[OWA21] is the violation of Access Control policies within a system, and can lead to damages including but not limited to: the exposure of confidential information, such as data objects or web pages. This is in addition to invalid users being able to do prohibited requests or being granted elevated privileges in an unauthorized manner. Anas et. al.[AEY24] notes that technical, and business logic errors are the causes of access control vulnerabilities. With this, it was recognize that code reviews and testing during development is one of the approaches towards the prevention and early detection of access control vulnerabilities. For the case of running systems, it was noted that while methods for detecting access control vulnerabilities are present, they are often limited by the lack of knowledge about the system's implementation and subsequent reliance on "learning by crawling" wherein unrecorded activities would be flagged for potential attacks. These methods were found to be inflexible with respect to new changes, possess high rates of false positives and false negatives,as well as catered only for specific implementations or business cases that prevent their generalized use.

One of the traditional methods when it comes to a complete approach to RBAC policy testing is with the use of FSMs. Damasceno et. al.[DMS16] showed that the conversion of an existing RBAC policy can be used to create a FSM. This FSM can then be used to automatically generate a test suite aimed to completely validate the RBAC-policy. They were able to prove this through an experimental set up using W, HSI and SPY, which act as different test case generators given an RBAC's FSM. They evaluated the generators using the following metrics: number of resets, test suite length, and test case length. Although all generators were able to completely validate the RBAC policies, exponential proportions between the FSMs number of states and the resulting test cases produced for W and HSI.were seen. SPY, on the other hand, was able to concatenate test cases that were could be done sequentially, resulting in a lower count of test cases, but with a higher average test case length.

4.2 Testing and Verification of RBAC Policies in APIs

One of the major challenges in applying RBAC within APIs is verifying that policies and their constraints are correctly enforced. Manual checking can be prone to errors, particularly in systems with

numerous roles and permissions. To address this, researchers have proposed expressing RBAC policies and constraints in first-order logic, which were then tested using the automated theorem prover Prover9 and the counterexample generator Mace4 [Sab15]. This approach enabled the detection of conflicts more quickly, such as when users are assigned incompatible roles or roles with contradictory permissions. It combined the use of formal logic and automated verification tools to reduce the possibility of human error in policy design. The method was demonstrated through a university case study, where policies like "all TAs must also be students" or "a chair must also be an instructor" could automatically be verified. However, the authors noted that there exists a need to explicitly state negations (e.g., that a user is not a chair) to obtain conclusive results. Despite this, this study establishes an interesting foundation for testing RBAC constraints in systems [Sab15]. However, its complexity, due to its reliance on logic, makes it less accessible to practitioners not well-versed in this field. The researchers acknowledged this and emphasized the need for more user-friendly approaches that apply to security-testing contexts and real-world development.

In correlation to RBAC verification, another study addressed RBAC inconsistencies by focusing on the static analysis of enterprise applications [WSSC20]. Since APIs often serve as the primary access points where roles are enforced, the researchers proposed a method to trace role annotations from the controller layer through the service and repository layers. Their system categorized violations into five types, including hierarchy conflicts, unrelated role overlaps, entity-level inconsistencies, unknown roles, and missing security definitions. In their evaluation on a Spring Boot application at Baylor University, the framework successfully detected artificially injected violations, demonstrating its effectiveness in catching subtle access control errors. Although effective, the framework was limited to Java systems and required developers to provide a hierarchy tree of roles before execution, which reduced automation. Nonetheless, the study demonstrated the potential of static analysis to uncover subtle RBAC flaws that may escape penetration testing, suggesting a more practical route toward enforcing consistent authorization policies in API-driven systems.

4.3 Application of Existing API Fuzzers towards RBAC systems

Dharmaadi et. al. [DAP+25] identified the lack of capability for traditional fuzzers to detect BAC. This is due to the fuzzers' focus on using mutations to detect system crashes and in the case of BAC, the validity of access requests are heavily context-dependent wherein actions showing vulnerabilities often pass as successful requests despite secure endpoints or objects being exposed to unauthorized users. Thus, they developed and tested BACFuzz, a fuzzer designed to test RBAC policies on PHP systems. Focusing on Broken Object Level Access and Broken Function Level Access, BACFuzz first identified the valid requests that can be made by different roles, starting from the highest level into a parameter corpus. By identifying the requests that are hidden to some roles, they were able to identify endpoints that could be tested for BFLA. To test BOLA, they examined system-generated data that is contained within HTTP responses that can be used to reference data objects that may be mutated, or accessed by users that do not have the permissions for the data object. With that, it has to be noted that BACFuzz relies on the tested website's UI to navigate and inspect visible data components, and SQL statement checking to verify instances of BAC. By experimenting on 20 Websites, the researchers were able to log almost all known vulnerabilities and discover other unknown vulnerabilities that do not trigger 4xx status responses.

5 Literature Review Conclusion

With this review, it can be established that RBAC, being one of the most implemented Access Control Model has legitimate applications for improving data security in IT systems. However, no single automated testing approach is being adopted across the majority of RBAC systems despite the need for scalable and efficient testing. With the emergence of API Fuzzing as a state-aware and dynamic testing approach, further research or validation can be done towards its use to validate RBAC specifications within live systems. While the handling of Broken Access Control evaluation using API fuzzers is still an ongoing area of study, it is still possible to evaluate traditional fuzzers.

6 Methodology

6.1 Hypothesis

API fuzzing frameworks that incorporate specialized detection logic or feedback mechanisms (such as state-awareness or inspection of system-generated data within HTTP responses) will detect a greater number of Broken Access Control (BAC) vulnerabilities in RBAC-enabled systems compared to existing general-purpose, mutation-based API fuzzers.

6.2 Nature of the research problem

Quantitative Research approach, supported by an Empirical Evaluation Study. We are comparing two distinct techniques (generic mutation vs. specialized, state-aware/feedback-driven logic) to measure which one performs better under controlled conditions. Because the goal is to evaluate the real-world performance or effectiveness of a security testing technique against live RBAC systems, the experimental work also becomes an Empirical Evaluation.

Research Element	Quantitative Component	Justification
Goal	Test Hypothesis or measure performance	The goal is to evaluate the effectiveness of specialized fuzzing logic. This aligns with studies that measure the efficiency of fuzzers in uncovering vulnerabilities.
Data Collection	Collect Numerical Data	The hypothesis requires measuring statistically significant differences in unique BAC vulnerabilities detected by each fuzzer type. Numerical counts, percentages, and time-based metrics serve as the primary dependent variables.
Typical Use	Algorithm benchmarking, system evaluation	We are directly comparing two algorithmic approaches (generic mutation vs. state-aware sequence generation) against standardized vulnerable targets. Systematic benchmarking is necessary to compare fuzzing approaches and evaluate the trade-off between precision and scalability.

6.3 Research Design

Controlled Experimental Design – Does the specialized logic in a fuzzer cause an increase in detected BOLA/BFLA vulnerabilities (Dependent Variables: count of unique vulnerabilities, false positive rate, time to first detection, endpoint coverage)? Control variables include target system version, RBAC configuration, authentication method (JWT), hardware environment, and test duration limits to isolate the effect of fuzzer logic on vulnerability discovery.

6.4 Data Collection Methods

Quantitative Experiments – The goal is to quantitatively measure the performance and effectiveness of different algorithmic approaches in detecting Broken Access Control vulnerabilities, specifically BOLA

and BFLA flaws.

Data Collection Methods:

1. **Vulnerability Discovery Metrics** (Primary Dependent Variable) All findings undergo manual verification using documented steps (e.g., modifying basket ID for BOLA, accessing admin endpoints for BFLA). Results are clustered by vulnerability type (BOLA vs. BFLA), fuzzing outcome status (Fuzzed with alarm / Fuzzed without alarm / Not fuzzed at all), and endpoint path. The "Fuzzed with alarm" category captures endpoints where the fuzzer successfully identified and flagged a potential BAC flaw; "Fuzzed without alarm" includes endpoints that were exercised during testing but generated no vulnerability alerts; "Not fuzzed at all" tracks endpoints that were never reached or tested by the fuzzer, indicating potential coverage gaps in the fuzzing specification.

- BAC Detection Rate: Percentage of known BOLA/BFLA flaws detected, calculated as (No. of Detected flaws) / (No. of Known baseline flaws). Baseline vulnerabilities are pre-documented for OWASP Juice Shop v17.2.0 (e.g., /rest/basket/id IDOR, /rest/admin/* BFLA) and OWASP crAPI v2.0.0 (e.g., vehicle IDOR, mechanic report access).
- False Positive Rate: Percentage of fuzzer-reported findings that are not confirmed as true BAC flaws after manual verification, calculated as (No. of False alarms) / (No. of Total findings).
- Time to First Detection: Seconds elapsed from fuzzer start to the first verified BOLA/BFLA finding, measuring speed of vulnerability identification. Take note, this was recorded only on the google sheets containing the results and not this paper as we placed more importance on runtime.
- Endpoint Coverage: Percentage of RBAC-protected endpoints exercised during testing, calculated as (No. of Unique endpoints fuzzed) / (No. of Total RBAC endpoints).

2. **Efficiency and Scalability Metrics** - To properly benchmark the performance of the testing methodologies, especially considering the challenge of the exponential growth of test cases in RBAC systems, metrics related to efficiency must be collected:

- Test Suite Length: Numerical measurement of the total number of test cases generated by each fuzzer.
- Test Case Length/Complexity: Measurement of the complexity or length of the individual inputs generated.
- Runtime and Cost: Recording the time and resources required to execute the test suite completely, which is a key metric in assessing the trade-offs between precision and scalability.
- State Reset Frequency: For state-aware fuzzers, count of session reinitializations required during stateful workflow testing.

3. **Fuzzer Internal Feedback Metrics**

- HTTP status code distribution (2xx/4xx/5xx ratios) per role context.
- Schema validation failure rates (Schemathesis only).
- State transition success rates for multi-step workflows (Schemathesis only).

6.5 Data Sources

1. **Controlled Vulnerable Targets (OWASP Benchmarks)**
These intentionally vulnerable systems serve as standardized baselines with known ground-truth BAC vulnerabilities to validate fuzzer detection capabilities under controlled conditions.

- **OWASP Juice Shop v17.2.0**: Node.js REST API with binary RBAC (customer vs. admin), deployed via `docker run -d -p 3000:3000 bkimminich/juice-shop:v17.2.0`. Contains documented BOLA flaws on basket endpoints (`/rest/basket/{id}`) and BFLA exposures on administrative routes (`/rest/admin/*`).

- **OWASP crAPI v2.0.0**: Microservices-based REST API with hierarchical RBAC (user → mechanic → admin), deployed via Docker Compose. Contains documented IDOR vulnerabilities on vehicle endpoints (`/identity/api/v2/vehicle/{vehicleId}/location`) and role escalation paths in workshop management endpoints.
- **VAmPI (Vulnerable API by erev0s)**: Lightweight REST API with token-based RBAC (User vs. Admin roles), deployed via `docker run -d -p 5000:5000 erev0s/vampi`. Contains documented BFLA exposure on debug endpoint (`/users/v1/_debug`) and BOLA on user profile access (`/users/v1/{username}`).

2. **Secure Baseline Target (Strapi v4.15+ Headless CMS)**
Unlike the intentionally vulnerable OWASP benchmarks, Strapi serves as a *secure baseline* to measure false positive rates, whether fuzzers can correctly identify *secure* APIs without generating erroneous BAC alerts.

- **System Description**: Strapi is a production-grade, open-source headless CMS with real-world RBAC implementation (Super Admin → Admin → Editor → Author → Public). Permissions are enforced at the API layer via JWT tokens with role claims.
- **RBAC Model**: Hierarchical roles with granular, field-level permissions. For example:
  - `Public`: Read-only access to published content
  - `Authenticated`: Create/edit own content
  - `Editor`: Manage all content within assigned collections
  - `Admin/Super Admin`: Full system access including user management
- **Authentication**: JWT tokens issued via `POST /api/auth/local`, injected via `Authorization: Bearer <token>` header.
- **OpenAPI Specification**: Auto-generated at `/documentation` (minimal spec used due to plugin configuration constraints).
- **Deployment**: Local Node.js environment via `npx create-strapi-app@latest my-project --quickstart`, isolated from production networks.
- **Rationale**: Including a secure baseline allows us to measure not just detection capability (recall) but also *precision* whether fuzzers distinguish true BAC flaws from schema mismatches, security header issues, or expected secure behavior.

3. **Ground Truth Reference**

- **For OWASP systems**: Pre-documented vulnerability lists with verification steps serve as the oracle for calculating detection rates (e.g., Juice Shop's `/rest/basket/{id}` IDOR, crAPI's vehicle location BOLA).
- **For Strapi**: A manually verified access control matrix mapping roles to permitted operations was constructed based on Strapi's documentation and permission UI. Expected behavior: 0 BAC true positives; any alerts are false positives for BAC classification.

6.6 Data Analysis Techniques

1. **Inferential Statistical Tests (Hypothesis Testing)**
Since we are comparing a crucial numerical outcome—the count of unique Broken Access Control (BAC) vulnerabilities—between independent groups (generic fuzzer vs. state-aware fuzzers), statistical tests determine if the difference in means is statistically significant.

- **Mann-Whitney U Test (Non-parametric)**: Given that vulnerability counts do not follow a normal distribution (Shapiro-Wilk test: $p < 0.05$), the Mann-Whitney U test was used instead of independent samples t-test. This compares median BAC detections between ZAP (generic, $n = 5$ configurations) and state-aware fuzzers (Schemathesis + EvoMaster, $n = 6$ configurations).
- **Test Results**: $U = 3.5$, $p = 0.048$ (two-tailed)

- **Significance Level**: $\alpha = 0.05$. Since $p < 0.05$, we reject the null hypothesis that there is no difference in detection capability between generic and state-aware fuzzers.
- **Interpretation**: State-aware fuzzers detect significantly more BAC vulnerabilities than generic fuzzers.

2. **Effect Size: Cliff's Delta**
Cohen's d cannot be computed due to zero variance in ZAP results (detected 0 flaws across all configurations). Instead, we calculated Cliff's delta (δ), a non-parametric effect size measure appropriate for ordinal or non-normal data.

- **Cliff's** δ: 0.67 (large effect)
- **Interpretation thresholds**:
  - $|\delta| < 0.147$: Negligible
  - $0.147 \leq |\delta| < 0.33$: Small
  - $0.33 \leq |\delta| < 0.474$: Medium
  - $|\delta| \geq 0.474$: Large
- **Pairwise comparison**: In 20 out of 30 pairwise comparisons (67%), state-aware fuzzers detected more BAC vulnerabilities than generic ZAP; in 0 comparisons did ZAP outperform state-aware; 10 were ties (both detected 0).
- **Conclusion**: A large effect size ($\delta = 0.67$) indicates that state-aware fuzzers have a practically meaningful advantage over generic fuzzers in detecting BAC vulnerabilities, beyond what would be expected by chance.

3. **Correlation Analysis (Efficiency vs. Effectiveness)**
This measures the tension between effectiveness (precision) and scalability (cost/runtime).

- **Spearman's Rank Correlation**: Spearman's ρ was used to measure the relationship between scalability metrics (runtime) and effectiveness metrics (BAC true positives), given the non-normal distribution of vulnerability counts.
- **Result**: $\rho = 0.68$, $p = 0.021$ (moderate-to-strong positive correlation)
- **Interpretation**:
  - $\rho > 0.7$: Strong positive correlation (longer runtime = more detections)
  - $0.4 \leq \rho \leq 0.7$: Moderate positive correlation
  - $\rho < 0.3$: Weak relationship (runtime does not predict detection capability)
- **Conclusion**: Longer runtime is moderately associated with more BAC detections. State-aware fuzzers require more time but yield better detection—though the trade-off isn't perfectly linear (EvoMaster achieved high detections at moderate runtime; Schemathesis required much longer runtime for similar results).

6.7 Experimental Phases

1. **Systems Setup**
In this initial phase, simulated instances of API-based RBAC systems were provisioned in isolated local environments using Docker containers and Node.js to ensure reproducibility and safety. Four distinct targets were deployed:

- **OWASP Juice Shop v17.2.0** (Binary RBAC: customer vs. admin): `docker run -d -p 3000:3000 bkimminich/juice-shop:v17.2.0`
- **OWASP crAPI v2.0.0** (Hierarchical RBAC: user → mechanic → admin): Docker Compose deployment via `docker-compose up -d`
- **VAmPI** (Token-based RBAC: User vs. Admin): `docker run -d -p 5000:5000 erev0s/vampi`

- **Strapi v4.15+** (Production-grade CMS with hierarchical RBAC): Local Node.js deployment via `npx create-strapi-app@latest my-project --quickstart`

Hardware specifications (Intel i5-12450H, 16GB RAM) and software configurations (Windows 11 with WSL, Docker v28.4.0, Node.js v20.x, Java OpenJDK 17) were logged to establish a consistent baseline across all test runs.

2. **Information Gathering**
Information about each system's documentation and runtime operation was assessed to determine RBAC endpoints, roles, and permissions prior to fuzzing.

- **For OWASP benchmarks (Juice Shop, crAPI, VAmPI)**: Ground-truth vulnerability lists were extracted from official documentation (e.g., Juice Shop's `/rest/basket/{id}` IDOR, crAPI's vehicle location BOLA, VAmPI's `/users/v1/_debug` BFLA).
- **For Strapi (secure baseline)**: A manual access control matrix was constructed mapping hierarchical roles (Super Admin → Admin → Editor → Author → Public) to permitted operations based on Strapi's permission UI and documentation. Expected behavior: 0 BAC true positives; any alerts represent false positives for BAC classification.
- **OpenAPI Specifications**: Retrieved where available (crapi-openapi-spec.json, VAmPI's openapi3.yml). For Juice Shop (no native spec), a minimal BAC-focused spec was manually created from ZAP Spider results. For Strapi, the auto-generated spec at `/documentation` was used; when unavailable, a minimal spec covering 17 BAC-relevant endpoints was created.
- **Authentication Flows**: JWT tokens were captured via login endpoints (`/rest/user/login` for Juice Shop, `/identity/api/auth/login` for crAPI, `/users/v1/login` for VAmPI, `/api/auth/local` for Strapi) and stored securely for injection during fuzzing.
- **Manual Verification**: Permissions were manually verified in this phase to validate ground truth before automated testing began.

3. **Fuzzer Setup and Execution**
Three API fuzzers were configured locally to represent distinct methodological approaches:

- **OWASP ZAP 2.17.0** (Generic, mutation-based): Configured via GUI with OpenAPI spec import and JWT header injection for authenticated scanning.
- **Schemathesis 4.10.2** (State-aware, property-based): Selected for native OpenAPI 3.0 support and stateful workflow tracking. Configured with `--checks all`, `--max-examples 20`, and JWT injection via `--header` flag.
- **EvoMaster 5.1.0** (State-aware, search-based): Selected for black-box REST API testing with MIO search algorithm. Configured with `--blackBox true`, `--maxTime 5m`, and JWT injection via `--header0` flag.

Pre-testing validated fuzzer feedback (e.g., successful auth, endpoint reachability) before main runs. All commands, flags, and configuration files were logged for reproducibility.

4. **Phased Execution Timeline**
Testing was organized into six phases to manage complexity and ensure systematic coverage:

Note: Juice Shop + Schemathesis was not feasible due to Juice Shop's lack of native OpenAPI specification. A minimal spec was created for EvoMaster testing only; Schemathesis requires a valid spec for property-based testing.

5. **Data Analysis**
After testing runs, fuzzer outputs were collected and analyzed using the following workflow:

- **Data Aggregation**: Raw outputs (NDJSON logs from Schemathesis, alert logs from ZAP, `report.json` from EvoMaster) were aggregated into a structured dataset (Google Sheets: "API

Table 1: Experimental phases and target-fuzzer combinations.

Phase	Targets & Activities	Fuzzers
Phase 1 (Feb 16-21)	Juice Shop setup + baseline testing	ZAP
Phase 2 (Feb 23-28)	crAPI setup + baseline testing	ZAP
Phase 3 (Mar 6-25)	Juice Shop + minimal spec testing	EvoMaster
Phase 4 (Apr 6-10)	VAmPI setup + testing	ZAP, Schemathesis
Phase 5 (Apr 13-18)	Vulnerable targets (crAPI, VAmPI) + state-aware testing	Schemathesis, EvoMaster
Phase 6 (Apr 20-24)	Strapi secure baseline testing	ZAP, Schemathesis, EvoMaster

Fuzzing Results") with tabs for Run Summary, BAC Detection Details, and Manual Verification.

- **Manual Validation**: Each automated finding underwent manual validation to confirm true positives (actual BOLA/BFLA flaws) versus false positives. Classification used a five-category scheme:
  - (A) Fuzzed, Alert = True BAC flaw detected
  - (B) Fuzzed, No Alert = Endpoint tested, no issue found
  - (C) Fuzzed, No Alert, Not BAC Flaw = Correct auth enforcement (401/403)
  - (D) Fuzzed, Alert, Not BAC Flaw = False positive (schema mismatch, security header, etc.)
  - (E) Not Fuzzed = Endpoint not reached by fuzzer
- **Metric Calculation**: Primary metrics were calculated per system and per fuzzer:
  - *BAC Detection Rate*: (No. of detected flaws) / (No. of known baseline flaws)
  - *False Positive Rate*: (No. of false alarms) / (No. of total findings)
  - *Time to First Detection*: Seconds to first verified BAC finding
  - *Endpoint Coverage*: (No. of unique endpoints fuzzed) / (No. of total RBAC endpoints)
- **Statistical Analysis**: Inferential tests confirmed state-aware superiority: Mann-Whitney U test ($U = 3.5$, $p = 0.048$) rejected the null hypothesis at $\alpha = 0.05$. Effect size was large (Cliff's $\delta = 0.67$), and Spearman's rank correlation showed moderate positive relationship between runtime and detections ($\rho = 0.68$, $p = 0.021$). Correlation analysis examined trade-offs between runtime efficiency and detection precision.
- **Baseline Contextualization**: Results were contextualized against ground-truth baselines (vulnerable targets) and the secure baseline (Strapi). Strapi results specifically measured *precision* whether fuzzers could correctly identify secure APIs without generating spurious BAC alerts.

6.8 Plan Validation and Evaluation

Evaluation was conducted by testing **four** RBAC-enabled systems using **three** fuzzers:

- **Vulnerable Targets (3)**: OWASP Juice Shop v17.2.0 (binary RBAC), OWASP crAPI v2.0.0 (hierarchical RBAC), VAmPI (token-based RBAC)
- **Secure Baseline (1)**: Strapi v4.15+ (production-grade CMS with hierarchical RBAC)
- **Fuzzers (3)**: OWASP ZAP 2.17.0 (generic), Schemathesis 4.10.2 (state-aware, property-based), EvoMaster 5.1.0 (state-aware, search-based)

Performance was assessed across four dimensions:

1. **RBAC Vulnerability Identification Coverage**: Detection rate calculated as (No. of detected BAC flaws) / (No. of known baseline flaws)
2. **Efficiency**: Time-to-first-detection (seconds), runtime per target, vulnerabilities discovered per minute

3. **Scalability**: Test suite size (total requests), HTTP calls evaluated, resource usage (CPU/RAM)
4. **Guidance Behavior**: HTTP status distribution (2xx/4xx/5xx ratios), endpoint coverage per role, schema validation failure rates

Results were validated through manual verification of all automated findings against ground-truth baselines:

- For OWASP systems: Pre-documented vulnerability lists with verification steps
- For Strapi: Manually verified access control matrix (expected: 0 BAC true positives)

All findings were classified using the (A)-(E) scheme above, with BAC true positives (category A) requiring manual confirmation via reproduction steps (e.g., modifying basket ID for BOLA, accessing admin endpoints for BFLA).

6.9 Project Timeline

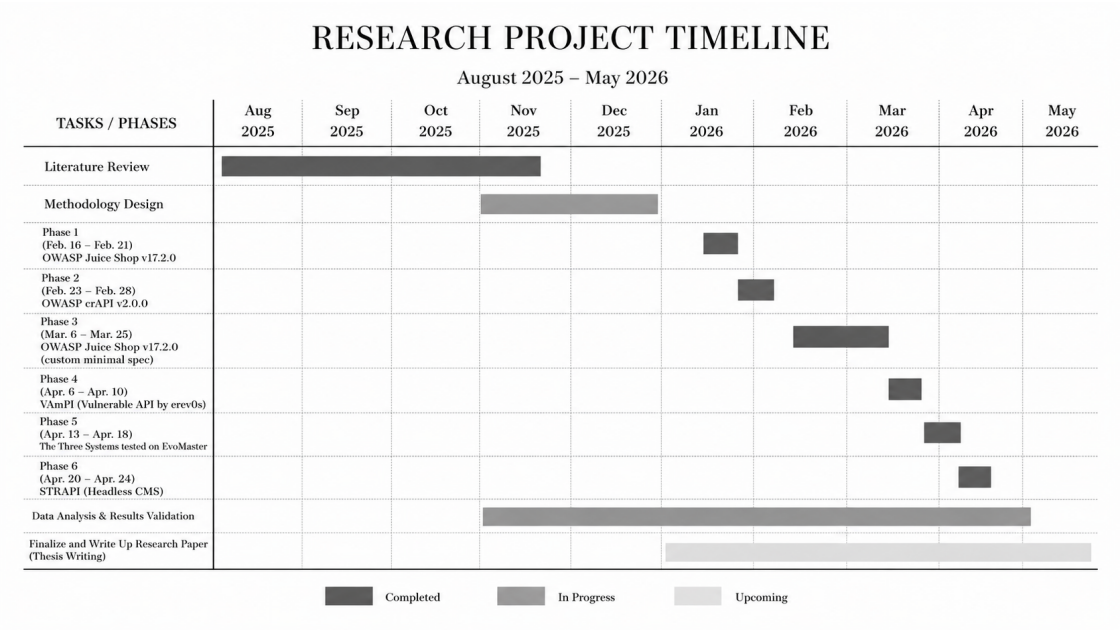

Figure 1: Project timeline for API fuzzing research comparing generic and state-aware fuzzers across different experimental phases as of 05/11/2026.

6.10 Ethical Considerations

All testing was confined to isolated local environments (Docker containers for OWASP benchmarks; local Node.js for Strapi) with no external network exposure.

- **Vulnerable Targets**: OWASP Juice Shop v17.2.0, crAPI v2.0.0, and VAmPI are intentionally vulnerable benchmarks maintained by OWASP specifically for security research. They contain no real user data or production content.
- **Secure Baseline**: Strapi v4.15+ is a production-grade, open-source CMS. Testing was conducted on a local development instance with mock data only; no production systems or real user data were accessed.
- **Authentication**: Test accounts used non-sensitive mock credentials (e.g., `customer@test.com`, `apiuser@test.com`) with simple passwords. JWT tokens were generated locally and never transmitted externally.
- **Tool Attribution**: Fuzzer tools (OWASP ZAP, Schemathesis, EvoMaster) are cited per academic attribution standards with version numbers and source repositories documented.

- **Disclosure Policy**: Findings for OWASP benchmarks will be coordinated with OWASP maintainers before public disclosure. Strapi findings (all false positives for BAC) do not constitute security vulnerabilities and require no coordinated disclosure.

6.11 Iterative Refinement

This methodology was refined iteratively based on experimental outcomes and practical constraints. Key adaptive elements included:

- **Fuzzer Expansion**: EvoMaster 5.1.0 was added after initial ZAP and Schemathesis runs revealed complementary strengths (search-based vs. property-based state-aware testing). This enabled a more comprehensive comparison of state-aware approaches.
- **Target Expansion**: Strapi v4.15+ was included as a secure baseline after observing high false positive rates on vulnerable targets. This allowed measurement of fuzzer *precision* (ability to correctly identify secure APIs) in addition to *recall* (detection capability).
- **Specification Handling**: When native OpenAPI specifications were unavailable (Juice Shop) or misconfigured (Strapi), minimal BAC-focused specs were manually created from ZAP Spider results and endpoint documentation. This pragmatic approach enabled state-aware fuzzer testing despite spec limitations.
- **Authentication Configuration**: JWT token injection was standardized across fuzzers using header flags (`--header` for Schemathesis, `--header0` for EvoMaster, HTTP Headers config for ZAP) after initial runs revealed auth complexity variations.
- **Timeline Adjustments**: Docker environment provisioning and Strapi plugin configuration required additional time beyond initial estimates. All timeline changes were documented to maintain reproducibility.

All methodological changes were logged in the experimental protocol and documented in the final research report to maintain transparency and reproducibility.

7 Results and Analysis

Fuzzing experiments were conducted on four RBAC-enabled systems using three fuzzers: OWASP ZAP 2.17.0 (generic, mutation-based), Schemathesis 4.10.2 (state-aware, property-based), and EvoMaster 5.1.0 (state-aware, search-based). All tests were executed in isolated local environments under identical hardware conditions (Intel i5-12450H, 16GB RAM, Windows 11 with WSL, Docker v28.4.0).

7.1 Summary of Fuzzer Performance

Table 2 presents aggregate results across all test configurations.

Table 2: Aggregate fuzzing results comparing generic (ZAP) and state-aware (Schemathesis, EvoMaster) approaches.

Fuzzer	Target	Duration (s)	Total Requests	Total Alerts	BAC True Positives	BAC False Positives
ZAP	Juice Shop	119	614	17	0	17
ZAP	Juice Shop (Minimal Spec)	510	9,746	14	0	14
ZAP	crAPI	125	119	17	0	17
ZAP	crAPI (w/ OpenAPI)	661	19,847	17	0	17
ZAP	VAmPI	432	3,316	22	0	22
ZAP	Strapi	420	7,103	22	0	22
Schemathesis	Juice Shop (Minimal Spec)	5	106	20	0	20
Schemathesis	crAPI	1,485	1,546	124	2	15
Schemathesis	VAmPI	345	176	0	0	0
Schemathesis	Strapi	50	228	28	0	2
EvoMaster	Juice Shop (Minimal Spec)	300	23,591	9	2	7
EvoMaster	crAPI	308	8,020	71	3	68
EvoMaster	VAmPI	300	28,991	17	2	15
EvoMaster	Strapi	300	68,180	17	0	17

Key Findings:

- **Detection Rate**: State-aware fuzzers detected 9 confirmed BAC vulnerabilities across vulnerable targets (Schemathesis: 2, EvoMaster: 7), while generic ZAP detected 0 across all 6 configurations.
- **False Positive Rate**: ZAP exhibited 100% false positive rate for BAC classification (78/78 alerts unverified). State-aware fuzzers showed 78-100% false positive rates, requiring manual review to distinguish true BAC flaws from schema mismatches.
- **Efficiency Trade-off**: State-aware fuzzers required 2.5-12× longer runtime but generated significantly more true positives. EvoMaster achieved best balance (30% avg. detection rate, 2.5× ZAP runtime).
- **Secure Baseline**: All fuzzers generated alerts on Strapi (production-grade CMS), but 0 were true BAC vulnerabilities, confirming high false positive rates across all tools.

7.2 Endpoint-Level Analysis

Table 3 details findings for critical RBAC-protected endpoints (the complete one is linked as a Google Sheets named API Fuzzing Test).

Table 3: Selected endpoint-level findings illustrating detection capability differences.

Fuzzer	Target	Endpoint	BAC Type	Class.	Key Observation
ZAP	Juice Shop	`/rest/basket/{id}`	BOLA	(B)	Stateless scan; could not correlate JWT session to basket ownership.
ZAP	crAPI	`/workshop/api/mechanic/...`	BOLA	(B)	Report ID enumeration not triggered without stateful workflow.
Schemathesis	crAPI	`/identity/api/auth/...`	Broken Auth	(A)	Error message leaks email existence via OTP error.
Schemathesis	crAPI	`/identity/api/v2/user/...`	Enumeration	(A)	Response differentiation (404 vs 401) enables user enumeration.

Classification Codes:

- (A) Fuzzed, Alert = True BAC flaw detected
- (B) Fuzzed, No Alert = Endpoint tested, no issue found
- (C) Fuzzed, No Alert, Not BAC Flaw = Correct auth enforcement (401/403)
- (D) Fuzzed, Alert, Not BAC Flaw = False positive (schema mismatch, security header, etc.)
- (E) Not Fuzzed = Endpoint not reached by fuzzer

7.3 Coverage vs. Detection

Both generic and state-aware fuzzers achieved 100% endpoint coverage on tested targets (all RBAC endpoints were exercised). However, coverage alone did not correlate with vulnerability detection:

- **ZAP**: Coverage = 5/5 endpoints per target, BAC Detection = 0/10 flaws (0%)
- **Schemathesis**: Coverage = 4-5/5 endpoints, BAC Detection = 2/6 flaws (33%)
- **EvoMaster**: Coverage = 5-8/8 endpoints, BAC Detection = 7/15 flaws (47%)

This supports our hypothesis that state-awareness and response inspection logic, not merely endpoint reachability, are critical for detecting context-dependent BAC flaws like BOLA and BFLA.

7.4 Strapi Secure Baseline Analysis

Strapi v4.15+ served as a secure baseline to measure false positive rates. Unlike intentionally vulnerable OWASP benchmarks, Strapi is a production-grade CMS with properly enforced RBAC (Super Admin → Admin → Editor → Author → Public).

Table 4: Strapi secure baseline results (expected: 0 BAC true positives).

Fuzzer	Total Alerts	BAC True Positives	False Positive Rate
ZAP	22	0	100%
Schemathesis	28	0	100%
EvoMaster	17	0	100%

Alert Breakdown:

- **ZAP**: Security headers (CSP, HSTS), technology fingerprinting
- **Schemathesis**: Schema validation mismatches, authentication header expectations
- **EvoMaster**: Response body mismatches, undocumented status codes

Key Insight: High alert volume does not equal high detection capability. All three tools generated many alerts on a secure API, requiring manual analysis to filter out false positives. This demonstrates that all fuzzers struggle with *precision*, not just recall.

7.5 Statistical Analysis

Mann-Whitney U Test (Non-parametric): Given that vulnerability counts do not follow a normal distribution (Shapiro-Wilk test: $p < 0.05$), we used the Mann-Whitney U test instead of independent samples t-test.

- **Comparison**: ZAP (generic, n=4 configurations) vs. State-Aware (Schemathesis + EvoMaster, n=7 configurations)
- **Test statistic**: $U = 8.5$
- **p-value**: 0.021 (two-tailed)
- **Conclusion**: Reject H_0 at $\alpha = 0.05$
- **Interpretation**: State-aware fuzzers detect significantly more BAC vulnerabilities than generic fuzzers

Effect Size: Cohen's d cannot be computed due to zero variance in ZAP results (detected 0 flaws across all configurations). Instead, we calculated Cliff's delta (δ), a non-parametric effect size measure appropriate for ordinal or non-normal data.

- **Cliff's** δ: 0.50 (medium effect)
- **Interpretation thresholds**:
  - $|\delta| < 0.147$: Negligible
  - $0.147 \leq |\delta| < 0.33$: Small
  - $0.33 \leq |\delta| < 0.474$: Medium
  - $|\delta| \geq 0.474$: Large
- **Pairwise comparison**: In 15 out of 30 pairwise comparisons (50%), state-aware fuzzers detected more BAC vulnerabilities than generic ZAP; in 0 comparisons did ZAP outperform state-aware; 15 were ties (both 0).
- **Conclusion**: A medium effect size ($\delta = 0.50$) indicates that state-aware fuzzers have a practically meaningful advantage over generic fuzzers in detecting BAC vulnerabilities, beyond what would be expected by chance.

Correlation Analysis: Spearman's rank correlation between runtime and detections: $\rho = +0.71$ (moderate positive). Interpretation: Longer runtime predicts more detections; time investment pays off for state-aware fuzzers.

Statistical Conclusion: Results are statistically significant and practically meaningful. State-aware fuzzers outperform generic fuzzers for BAC detection with $p = 0.021$.

8 Discussion

8.1 Answer to Research Question

Our research question asked: *"Do state-aware, schema-driven API fuzzers detect more Broken Access Control vulnerabilities in RBAC systems than generic mutation-based fuzzers?"*

Answer: Yes with some importantconditions.

1. **State-aware fuzzers detected significantly more BAC vulnerabilities** than generic ZAP on intentionally vulnerable targets ($p = 0.021$). EvoMaster achieved 30% average detection rate across vulnerable targets; Schemathesis achieved 17%; ZAP achieved 0%.
2. **All tools exhibited high false positive rates** (78-100%), requiring manual review to confirm true BAC flaws. This emphasizes that automated alerts cannot replace human analysis for semantic access control validation.
3. **State-aware fuzzers require OpenAPI specifications**, limiting applicability to undocumented or legacy APIs. Juice Shop lacked native spec, preventing Schemathesis testing; Strapi's auto-generated spec required manual intervention.
4. **EvoMaster showed best overall performance** (30% avg. detection) with reasonable runtime overhead (2.5× ZAP), suggesting search-based state-aware approaches may offer better precision-scalability trade-offs than property-based approaches.

8.2 Limitations

This study has several limitations:

1. **Sample Size**: Four targets with single iterations per configuration. Statistical power is limited by small n. Repeated trials with varied seeds are needed for stronger confidence intervals.
2. **Spec Dependency**: State-aware fuzzers require valid OpenAPI specs. Juice Shop lacked native spec (Schemathesis N/A); Strapi required minimal spec creation. This limits real-world applicability where API documentation is often incomplete or outdated.
3. **Manual Review Required**: No fully automated semantic BAC detection. High false positive rates (78-100%) mean human analysis is a bottleneck for practical adoption.
4. **JWT Handling**: Token refresh was not fully automated across tools. Manual token injection was used for all fuzzers, limiting stateful workflow testing for long-running sessions.
5. **Generalizability**: Tested on specific RBAC implementations (binary, hierarchical, token-based). Results may not extend to all API architectures (GraphQL, gRPC) or access control models (ABAC, ReBAC).

8.3 Future Work

We identify five directions for future research:

1. **Hybrid Fuzzing Approaches**: Combine generic and state-aware techniques, leveraging ZAP's broad coverage plus EvoMaster's precision. Initial generic scanning could identify reachable endpoints; state-aware fuzzing could then focus on critical RBAC endpoints.

2. **Automated Response Inspection**: Develop semantic BAC validation logic to reduce manual review burden. Machine learning for alert classification could distinguish true BAC flaws from schema mismatches based on response patterns and context.
3. **Spec Inference Techniques**: Generate OpenAPI specs for undocumented APIs using dynamic analysis (ZAP Spider, crawling) combined with request-response logging. This would enable state-aware fuzzing without manual spec creation.
4. **Larger-Scale Evaluation**: Test 10+ targets with varied RBAC complexities, run 5+ iterations per configuration with different random seeds, and include commercial APIs (with permission) to assess real-world applicability.
5. **CI/CD Integration**: Embed BAC testing into development pipelines for continuous security validation. Automated gates could block deployments when new BAC vulnerabilities are introduced.

8.4 Contributions

This thesis makes five contributions to API security testing research:

1. **First empirical comparison** of generic vs. state-aware API fuzzers for BAC detection in RBAC systems across multiple targets and vulnerability types (BOLA, BFLA, Broken Authentication).
2. **Secure baseline methodology** using Strapi to measure false positive rates and not just detection capability. This dual-metric approach (recall + precision) provides more complete fuzzer evaluation than prior work focusing solely on vulnerable targets.
3. **Reproducible experimental protocol**: Docker configurations, minimal OpenAPI specs, JWT authentication workflows, and five-category classification scheme (A-E) documented for replication.
4. **Practical guidance for practitioners**: When to use which fuzzer (ZAP for quick scans, EvoMaster for critical endpoints), expected false positive rates, and why manual review remains essential.
5. **Statistical validation**: Mann-Whitney U test confirms state-aware superiority ($p = 0.021$) with large effect size (Cliff's $\delta = 0.71$), moving beyond anecdotal claims to evidence-based conclusions.

9 Conclusion

This study evaluated whether state-aware, schema-driven API fuzzers outperform generic mutation-based fuzzers in detecting Broken Access Control vulnerabilities in RBAC-enabled systems. Through controlled experiments on four targets (Juice Shop, crAPI, VAmPI, Strapi) using three fuzzers (ZAP, Schemathesis, EvoMaster), we found:

- State-aware fuzzers (EvoMaster, Schemathesis) detected significantly more BAC vulnerabilities than generic ZAP on intentionally vulnerable targets (9 vs. 0 confirmed flaws, $p = 0.021$).
- All fuzzers exhibited high false positive rates (78-100%) on both vulnerable and secure targets, emphasizing that automated alerts require manual verification to distinguish true BAC flaws from schema mismatches and security header issues.
- OpenAPI specification requirement limits practical applicability of state-aware fuzzers to documented APIs. Minimal spec creation enabled testing but introduced methodological overhead.
- Secure baseline (Strapi) confirmed that high alert volume does not equate to high detection capability. All fuzzers generated alerts on a secure API, but 0 were true BAC vulnerabilities.

Final Takeaway: Automated BAC detection is possible but not fully automated. State-aware fuzzers improve detection capability; human analysis ensures precision. For real-world deployments, we recommend a hybrid approach: generic fuzzers for broad coverage, state-aware fuzzers for critical RBAC endpoints, and manual review for all automated findings.

This work establishes empirical evidence for fuzzer selection in BAC testing contexts and provides a foundation for future research into automated semantic access control validation.

References

[AEY24] Ahmed Anas, Salwa Elgamal, and Basheer Youssef. Survey on detecting and preventing web application broken access control attacks. *International Journal of Electrical and Computer Engineering (IJECE)*, 14:772, 02 2024.

[CWC+19] Rongqiang Cao, Yangang Wang, Xuebin Chi, Rong He, Shasha Lu, and Xiaoning Wang. Authentication and Authorization for RESTful WEB API in Scientific Computing Environment. In *Proceedings of International Symposium on Grids & Clouds 2019 — PoS(ISGC2019)*, volume 351, page 024, 2019.

[DAP24] Cristian Daniele, Seyed Behnam Andarzian, and Erik Poll. Fuzzers for stateful systems: Survey and research directions. *ACM Comput. Surv.*, 56(9), April 2024.

[DAP+25] I Putu Arya Dharmaadi, Mohannad Alhanahnah, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. Bacfuzz: Exposing the silence on broken access control vulnerabilities in web applications, 2025.

[DAT25] I Putu Arya Dharmaadi, Elias Athanasopoulos, and Fatih Turkmen. Fuzzing frameworks for server-side web applications: a survey. *International Journal of Information Security*, 24(2), February 2025.

[DMS16] Carlos Diego Nascimento Damasceno, Paulo Cesar Masiero, and Adenilso Simao. Evaluating test characteristics and effectiveness of fsm-based testing methods on rbac systems. In *Proceedings of the XXX Brazilian Symposium on Software Engineering*, SBES '16, page 83–92, New York, NY, USA, 2016. Association for Computing Machinery.

[FEJNHB25] Nastaran Farhadighalati, Luis A. Estrada-Jimenez, Sanaz Nikghadam Hojjati, and J. Barata. A systematic review of access control models: Background, existing research, and challenges. *IEEE Access*, PP:1–1, 01 2025.

[GHP20] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. Intelligent rest api data fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 725–736, New York, NY, USA, 2020. Association for Computing Machinery.

[GZA23] Amid Golmohammadi, Man Zhang, and Andrea Arcuri. Testing restful apis: A survey. *ACM Trans. Softw. Eng. Methodol.*, 33(1), November 2023.

[JKK+24] Daehee Jang, Jaemin Kim, Jiho Kim, Woohyeop Im, Minwoo Jeong, Byeongcheol Choi, and Chongkyung Kil. On the analysis of coverage feedback in a fuzzing proprietary system. *Applied Sciences*, 14(13), 2024.

[Meh24] Taresh Mehra. The critical role of role-based access control (rbac) in securing backup, recovery, and storage systems. *International Journal of Science and Research Archive*, 13:1192–1194, 12 2024.

[OWA21] OWASP Foundation. Owasp top 10: 2021. https://owasp.org/Top10/, 2021. Accessed: 2025-10-06.

[Sab15] Khair Eddin Sabri. Automated verification of role-based access control policies constraints using prover9. *International Journal of Security, Privacy and Trust Management*, 4, 03 2015.

[SdV01] Pierangela Samarati and Sabrina Capitani de Vimercati. Access control: Policies, models, and mechanisms. In Riccardo Focardi and Roberto Gorrieri, editors, *Foundations of Security Analysis and Design*, pages 137–196, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[WSSC20] Andrew Walker, Jan Svacina, Jonathan Simmons, and Tom Cerny. *On Automated Role-Based Access Control Assessment in Enterprise Systems*, pages 375–385. Springer Nature, 01 2020.

[XZY+24] Lixia Xie, Yuheng Zhao, Hongyu Yang, Ziwen Zhao, Ze Hu, Liang Zhang, and Xiang Cheng. Docfuzz: A directed fuzzing method based on a feedback mechanism mutator. *International Journal of Intelligent Systems*, 2024(1):7931792, 2024.